One consideration for the choosing the value of the key is where you might encounter it out of context. For example, you may see it in a web UI or a log file as you try to debug why the system did something wrong. If your key is a database identifier (a number or a UUID), it'll be unremarkable and hard to distinguish.

To deal with this issue, add some context directly to the key itself. For example, if an order has the ID 1234, an idempotency key like `"idempotency_key-order-1234"` would be very obvious in log files, web UIs, and more. If you can't do this, or have some other constraints on what the key should be, you should store the value in the database along with the record. This will provide an unambiguous accounting of what key was used.

Let's add an idempotency key to `request_fulfillment`. The fulfillment service will look for it in the metadata passed in the API call:

snapshots/3-4/sidekiq-book/app/services/order_creator.rb

```
    def request_fulfillment(order)
      fulfillment_metadata = {}
      fulfillment_metadata[:order_id] = order.id
»     fulfillment_metadata[:idempotency_key] = "idempotency_key-order-#{order.id}"
      fulfillment.request_fulfillment(
        order.user.id,
        order.address,
```

You can see the advantage when the service supports idempotency keys: this was a one-line change. You also didn't have to consult old Star Trek episodes to help train your brain to think fourth-dimensionally, either!

You addressed the email service and the fulfillment service, but the payments service has the same potential failure modes. As an exercise for you, try to make that call idempotent. Have a look at `PaymentsServiceWrapper` to see what options you have to fix it.

Preventing Retries if You Cannot Make the Call Idempotent